

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE CIENCIAS FÍSICO MATEMÁTICO



UA: Programación Avanzada para ciberseguridad

Reto Fase 1

Documentación

NOMBRE DE LA MAESTRA: [Perla Marlene Viera González]

Estudiantes

Carlos Adrián Ramos Zacarías

Marcos Adrián González Soto

Anateyssi Hernandez Meza

Miguel Eduardo Flores Salazar

Axel Simón Moreno Lozano

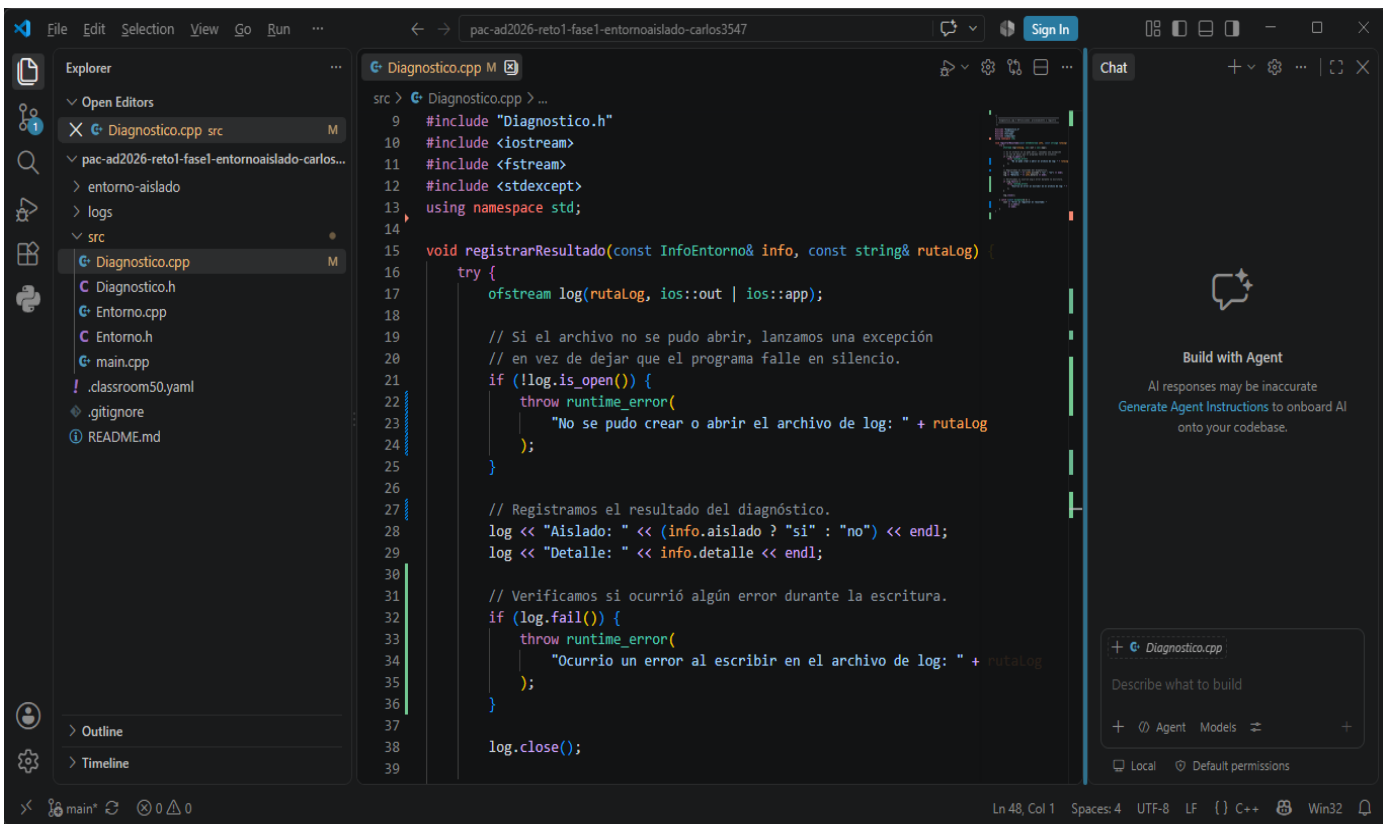
FECHA DE ENTREGA: [30 DE AGOSTO DEL 2026]

Documentación

Tabla de aportaciones en el proyecto

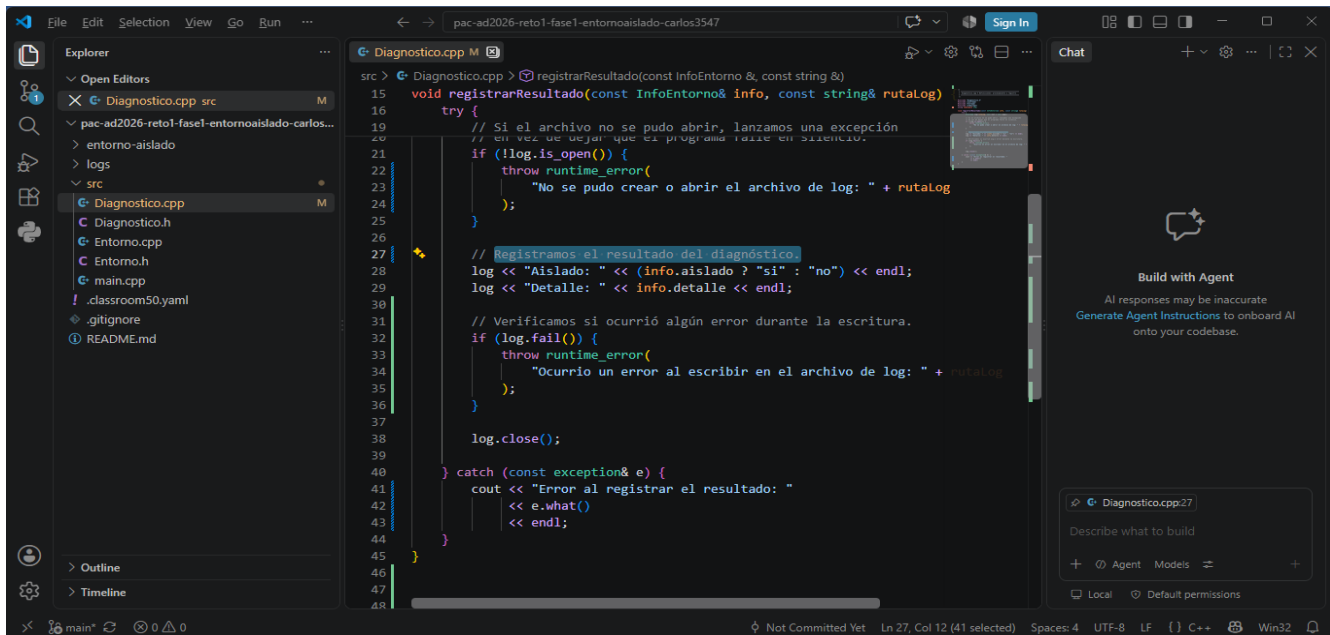
Integrantes	Aportación
Carlos Adrián Ramos Zacarías Script diagnostico.cpp	Lo que hice fue Registramos el resultado del diagnóstico. Verificamos si ocurrió algún error durante la escritura.
Marcos Adrián González Soto Script diagnostico.h	Cambié el struct InfoEntorno agregando dos campos: datoRed y datoSistema.
Anateyssi Hernandez Meza Entorno.h	Modifique el apartado de entorno h, lo que hice fue solo declare las funciones como la de estaEnEntorboAislado, y las extras que fue obtenerDatosRed y obtenerDatosDeSistema. Las funciones extra son para detectar si se aisló del internet y la otra para revisar lo de las configuraciones de Virtualbox o lo que usen de la máquina.
Miguel Eduardo Flores Salazar Script encorno.cpp	Se implementaron las funciones de obtenerDatosDeSistema () y obtenerDatoDeRed () que Anateyssi agrego en Entorno.h, agreguéconvertirAMinusculas () como función auxiliar y esa no estaba declarada y modifiqué la implementación de estaEnEntorboAislado (), porque originalmente solo regresaba false y ahora obtiene los datos del sistema y revisa mediante un for y un if si aparecen indicadores de virtualización
Axel Simón Moreno Lozano Script main.cpp	Se integra el flujo principal del programa combinando los módulos Entorno y Diagnostico. Se añade la recolección de datos, evaluación del aislamiento, manejo de memoria dinámica con punteros y el control de excepciones para abortar la accion real.

Archivo Diagnostico.cpp



```
src > Diagnostico.cpp M
9  #include "Diagnostico.h"
10 #include <iostream>
11 #include <fstream>
12 #include <stdexcept>
13 using namespace std;
14
15 void registrarResultado(const InfoEntorno& info, const string& rutaLog) {
16     try {
17         ofstream log(rutaLog, ios::out | ios::app);
18
19         // Si el archivo no se pudo abrir, lanzamos una excepción
20         // en vez de dejar que el programa falle en silencio.
21         if (!log.is_open()) {
22             throw runtime_error(
23                 "No se pudo crear o abrir el archivo de log: " + rutaLog
24             );
25         }
26
27         // Registramos el resultado del diagnóstico.
28         log << "Aislado: " << (info.aislado ? "si" : "no") << endl;
29         log << "Detalle: " << info.detalle << endl;
30
31         // Verificamos si ocurrió algún error durante la escritura.
32         if (log.fail()) {
33             throw runtime_error(
34                 "Ocurrió un error al escribir en el archivo de log: " + rutaLog
35             );
36         }
37
38         log.close();
39     }
```

Como tal al script le agregue el resultado del diagnóstico para que lo pueda guardar como un log en memoria

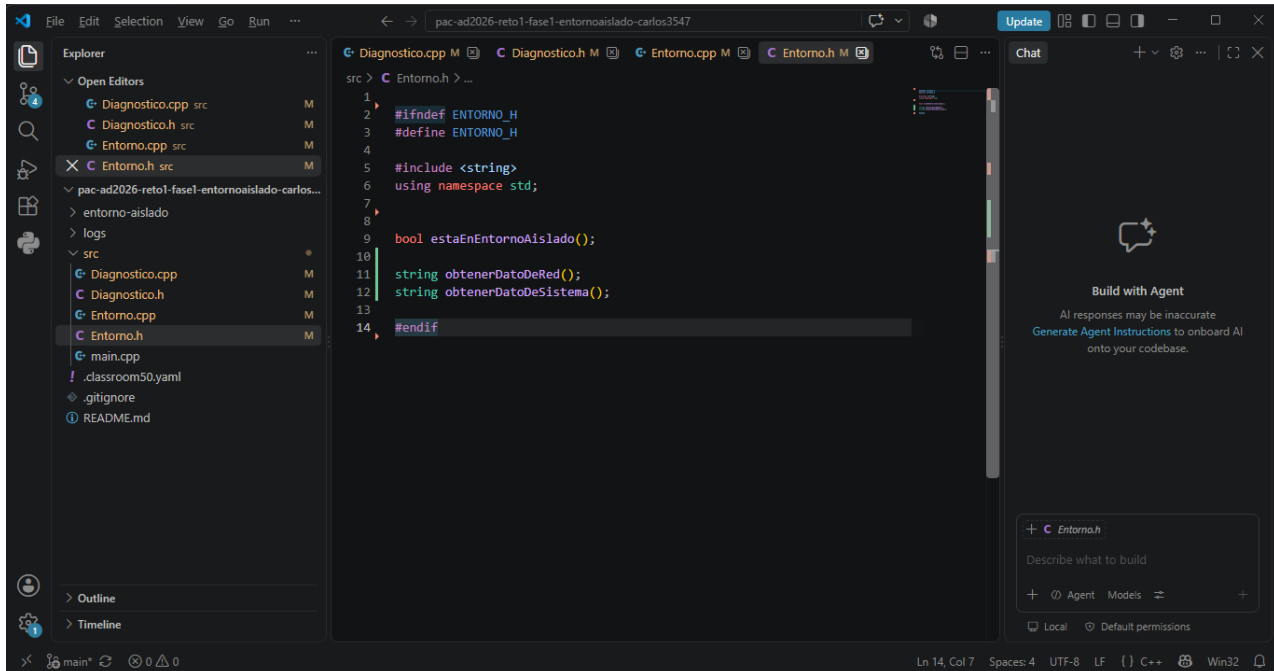


```
src > Diagnostico.cpp M
15 void registrarResultado(const InfoEntorno& info, const string& rutaLog) {
16     try {
17         // Si el archivo no se pudo abrir, lanzamos una excepción
18         // en vez de dejar que el programa falle en silencio.
19         if (!log.is_open()) {
20             throw runtime_error(
21                 "No se pudo crear o abrir el archivo de log: " + rutaLog
22             );
23         }
24
25         // Registramos el resultado del diagnóstico.
26         log << "Aislado: " << (info.aislado ? "si" : "no") << endl;
27         log << "Detalle: " << info.detalle << endl;
28
29         // Verificamos si ocurrió algún error durante la escritura.
30         if (log.fail()) {
31             throw runtime_error(
32                 "Ocurrió un error al escribir en el archivo de log: " + rutaLog
33             );
34         }
35
36         log.close();
37     } catch (const exception& e) {
38         cout << "Error al registrar el resultado: "
39              << e.what()
40              << endl;
41     }
42 }
```

Agregue la verificamos si ocurrió algún error durante la escritura

Entorno h

Solo modifique el apartado de entorno h, lo que hice fue solo declare las funciones como la de estaEnEntorboAislado, y las extras que fue obtenerDatosRed y obtenerDatosDeSistema

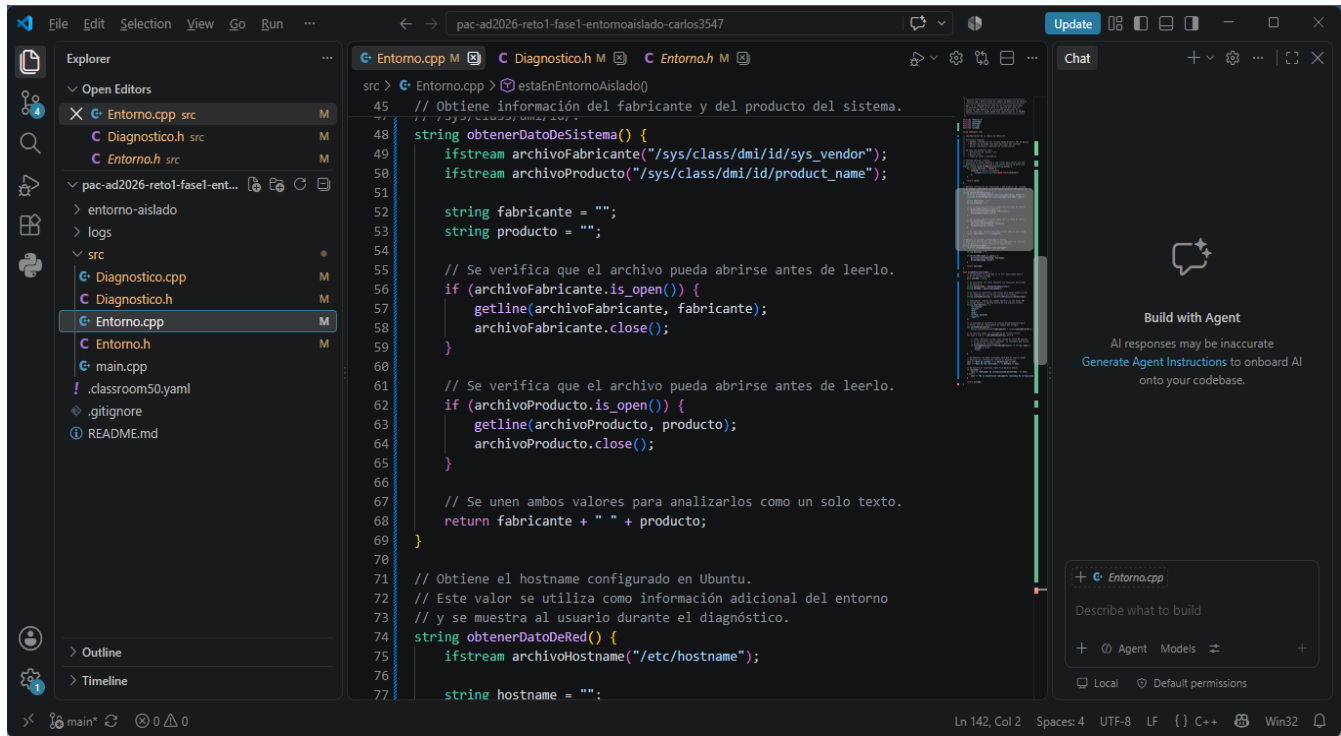


```
1  #ifndef ENTORNO_H
2  #define ENTORNO_H
3
4  #include <string>
5  using namespace std;
6
7
8
9  bool estaEnEntornoAislado();
10
11 string obtenerDatosRed();
12 string obtenerDatosDeSistema();
13
14 #endif
```

Las funciones extra son para detectar si se aisló del internet y la otra para revisar lo de las configuraciones de Virtualbox o lo que usen de la máquina.

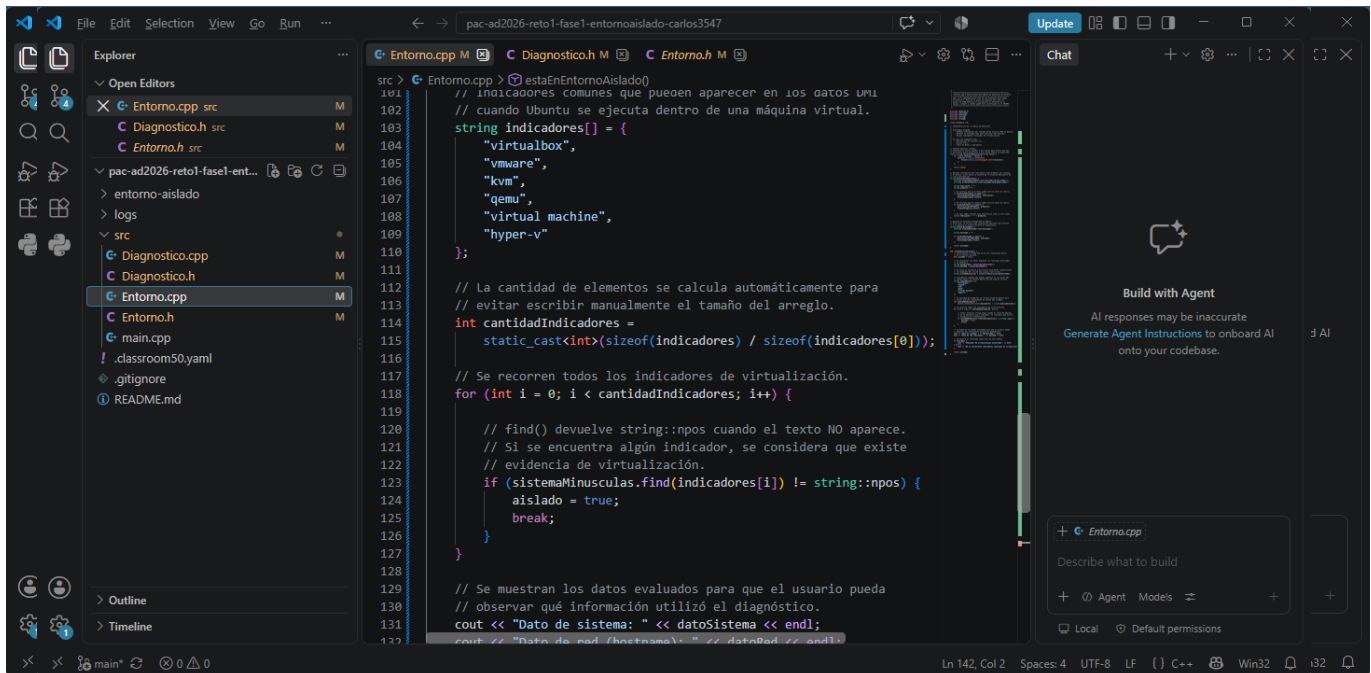
Entorno.cpp

Se implementaron las funciones de obtenerDatoDeSistema () y obtenerDatoDeRed () que Anateyssi agrego en Entorno.h



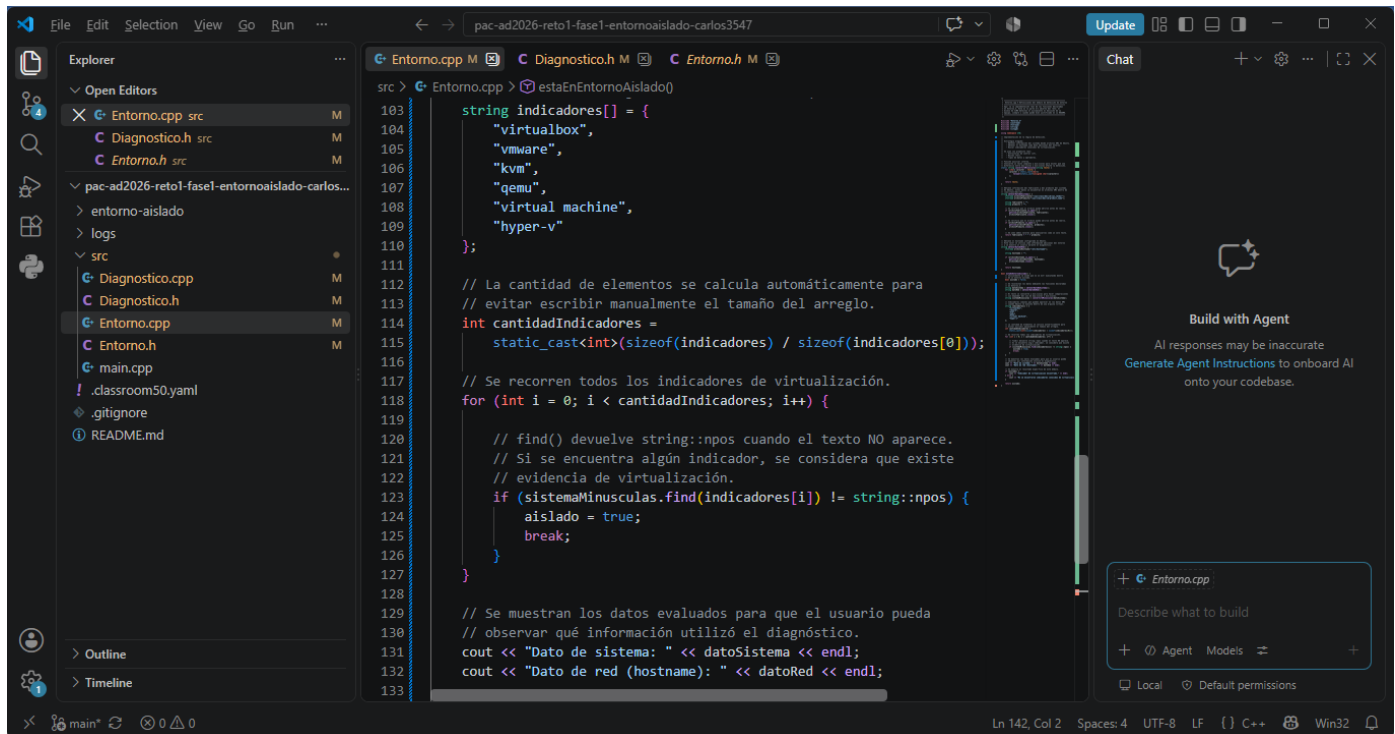
```
src > Entorno.cpp > estaEnEntornoAislado()
45 // Obtiene información del fabricante y del producto del sistema.
46 // sys/class/dmi/id/sys_vendor
47 // sys/class/dmi/id/product_name
48 string obtenerDatoDeSistema() {
49     ifstream archivoFabricante("/sys/class/dmi/id/sys_vendor");
50     ifstream archivoProducto("/sys/class/dmi/id/product_name");
51
52     string fabricante = "";
53     string producto = "";
54
55     // Se verifica que el archivo pueda abrirse antes de leerlo.
56     if (archivoFabricante.is_open()) {
57         getline(archivoFabricante, fabricante);
58         archivoFabricante.close();
59     }
60
61     // Se verifica que el archivo pueda abrirse antes de leerlo.
62     if (archivoProducto.is_open()) {
63         getline(archivoProducto, producto);
64         archivoProducto.close();
65     }
66
67     // Se unen ambos valores para analizarlos como un solo texto.
68     return fabricante + " " + producto;
69 }
70
71 // Obtiene el hostname configurado en Ubuntu.
72 // Este valor se utiliza como información adicional del entorno
73 // y se muestra al usuario durante el diagnóstico.
74 string obtenerDatoDeRed() {
75     ifstream archivoHostname("/etc/hostname");
76
77     string hostname = "";
```

agregué convertirAMinusculas () como función auxiliar y esa no estaba declarada.



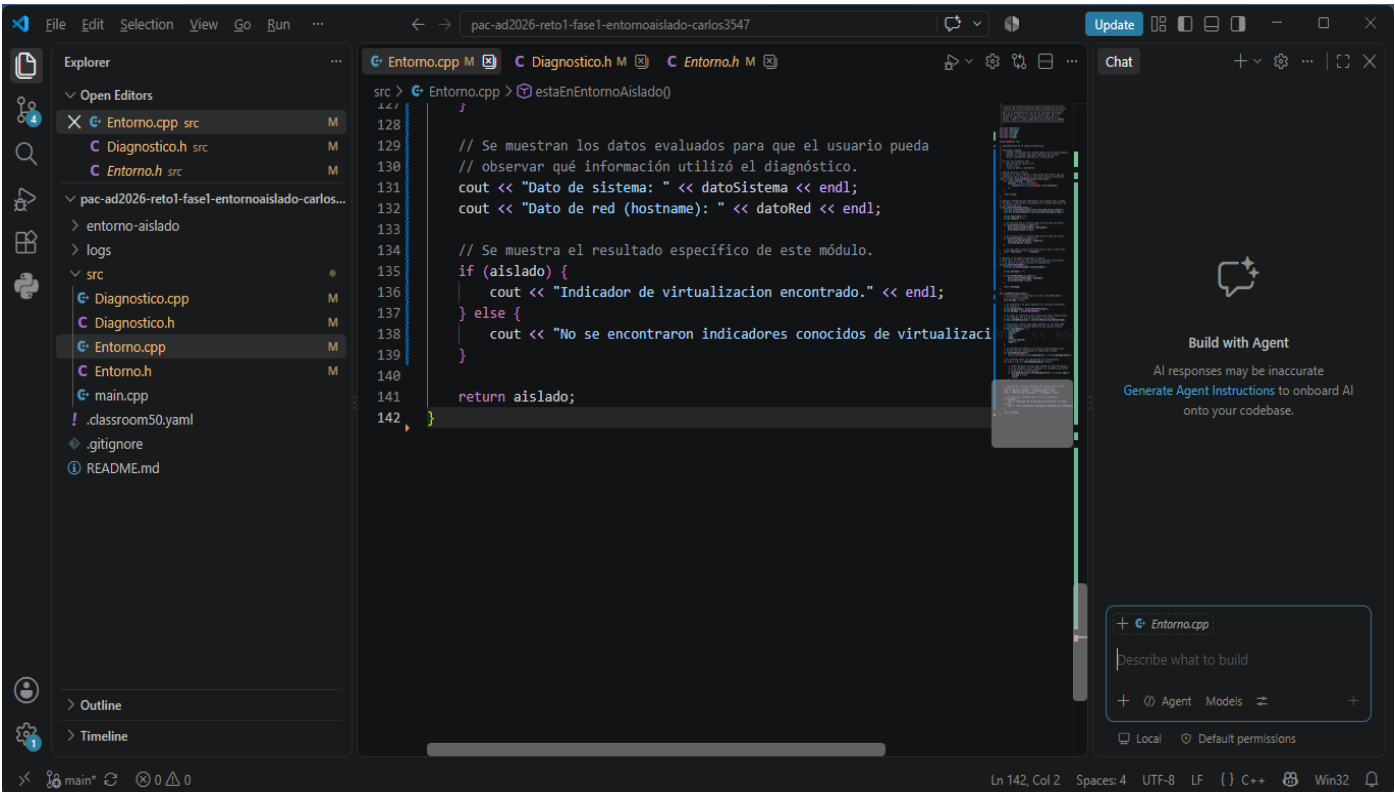
```
src > Entorno.cpp > estaEnEntornoAislado()
181 // Indicadores comunes que pueden aparecer en los datos UMI
182 // cuando Ubuntu se ejecuta dentro de una máquina virtual.
183 string indicadores[] = {
184     "virtualbox",
185     "vmware",
186     "kvm",
187     "qemu",
188     "virtual machine",
189     "hyper-v"
190 };
191
192 // La cantidad de elementos se calcula automáticamente para
193 // evitar escribir manualmente el tamaño del arreglo.
194 int cantidadIndicadores =
195     static_cast<int>(sizeof(indicadores) / sizeof(indicadores[0]));
196
197 // Se recorren todos los indicadores de virtualización.
198 for (int i = 0; i < cantidadIndicadores; i++) {
199
200     // find() devuelve string::npos cuando el texto NO aparece.
201     // Si se encuentra algún indicador, se considera que existe
202     // evidencia de virtualización.
203     if (sistemaMinusculas.find(indicadores[i]) != string::npos) {
204         aislado = true;
205         break;
206     }
207 }
208
209 // Se muestran los datos evaluados para que el usuario pueda
210 // observar qué información utilizó el diagnóstico.
211 cout << "Datos de sistema: " << datoSistema << endl;
212
213 cout << "Datos de red: " << datoRed << endl;
```

Modifiqué la implementación. de esta `EnEntornoAislado()`, porque originalmente solo regresaba false y ahora obtiene los datos del sistema y revisa mediante un for y un if si aparecen indicadores de virtualización.



The screenshot shows the Visual Studio Code editor with the file `Entorno.cpp` open. The code defines the `estaEnEntornoAislado()` function. It starts by defining an array of strings `indicadores` containing "virtualbox", "vmware", "kvm", "qemu", "virtual machine", and "hyper-v". It then calculates the number of indicators and iterates through them using a `for` loop. Inside the loop, it uses `find` to check if any indicator is present in the system's output. If found, it sets `aislado = true` and breaks the loop. Finally, it prints system and network data and returns the `aislado` status.

```
src > Entorno.cpp > estaEnEntornoAislado()
103 string indicadores[] = {
104     "virtualbox",
105     "vmware",
106     "kvm",
107     "qemu",
108     "virtual machine",
109     "hyper-v"
110 };
111
112 // La cantidad de elementos se calcula automáticamente para
113 // evitar escribir manualmente el tamaño del arreglo.
114 int cantidadIndicadores =
115     static_cast<int>(sizeof(indicadores) / sizeof(indicadores[0]));
116
117 // Se recorren todos los indicadores de virtualización.
118 for (int i = 0; i < cantidadIndicadores; i++) {
119
120     // find() devuelve string::npos cuando el texto NO aparece.
121     // Si se encuentra algún indicador, se considera que existe
122     // evidencia de virtualización.
123     if (sistemaMinusculas.find(indicadores[i]) != string::npos) {
124         aislado = true;
125         break;
126     }
127 }
128
129 // Se muestran los datos evaluados para que el usuario pueda
130 // observar qué información utilizó el diagnóstico.
131 cout << "Dato de sistema: " << datoSistema << endl;
132 cout << "Dato de red (hostname): " << datoRed << endl;
133 }
```

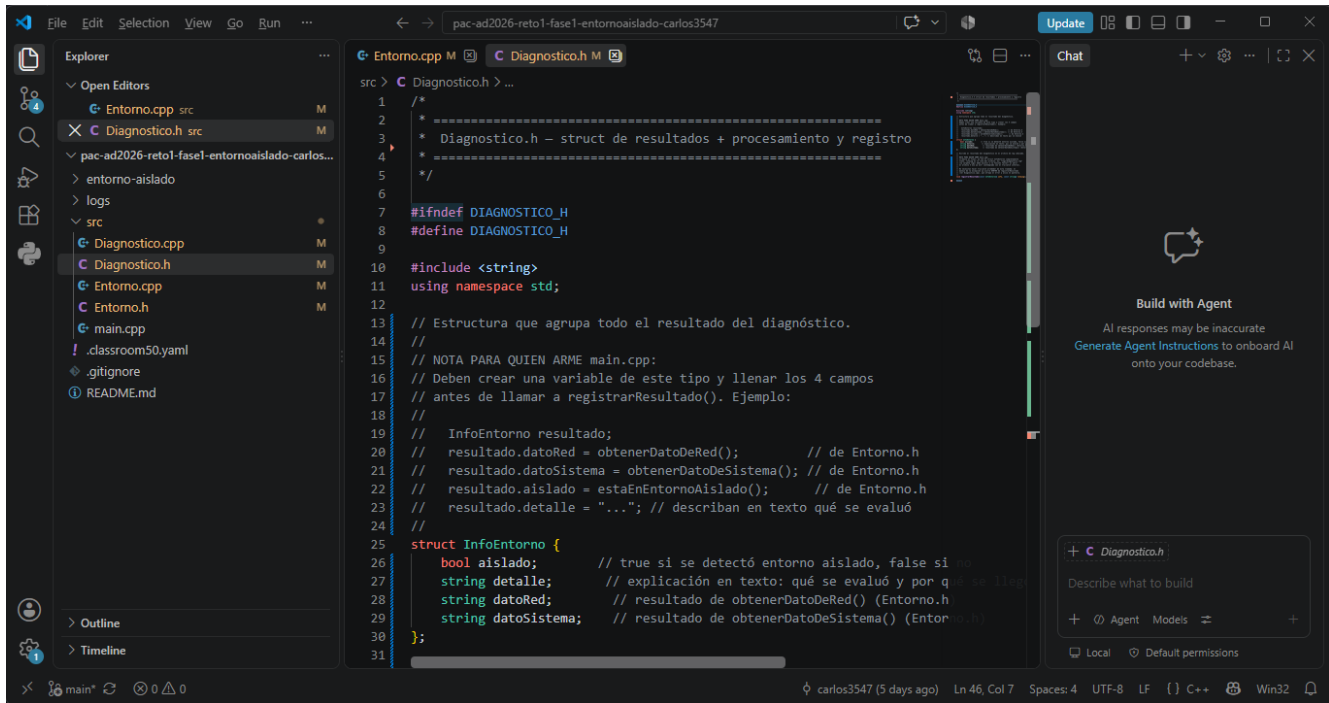


This screenshot shows the continuation of the `estaEnEntornoAislado()` function. It prints the system and network data, then checks if the system is isolated based on the `aislado` variable. If `aislado` is true, it prints "Indicador de virtualizacion encontrado."; otherwise, it prints "No se encontraron indicadores conocidos de virtualizacion". Finally, it returns the `aislado` status.

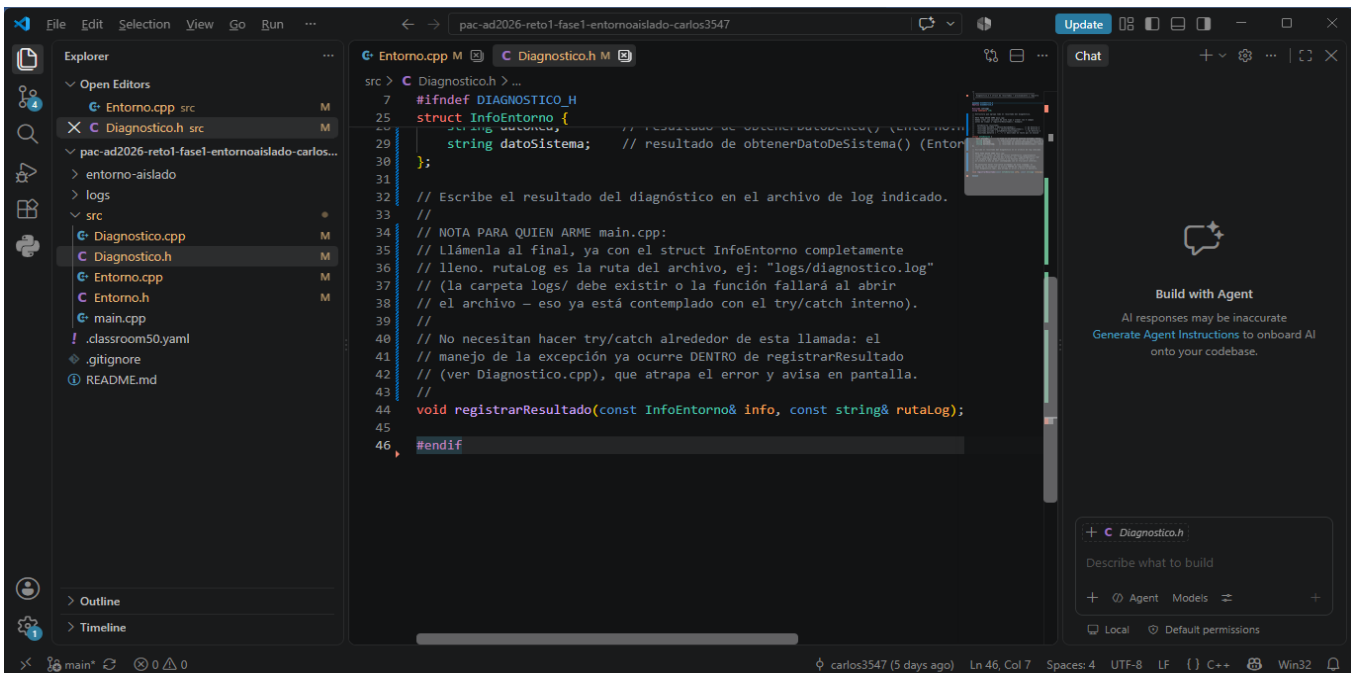
```
src > Entorno.cpp > estaEnEntornoAislado()
128 }
129
130 // Se muestran los datos evaluados para que el usuario pueda
131 // observar qué información utilizó el diagnóstico.
132 cout << "Dato de sistema: " << datoSistema << endl;
133 cout << "Dato de red (hostname): " << datoRed << endl;
134
135 // Se muestra el resultado específico de este módulo.
136 if (aislado) {
137     cout << "Indicador de virtualizacion encontrado." << endl;
138 } else {
139     cout << "No se encontraron indicadores conocidos de virtualizacion" << endl;
140 }
141
142 return aislado;
143 }
```

Diagnostico h

Cambié el struct InfoEntorno agregando dos campos: datoRed y datoSistema.



```
1  /*
2  * Diagnostico.h - struct de resultados + procesamiento y registro
3  *
4  */
5
6
7  #ifndef DIAGNOSTICO_H
8  #define DIAGNOSTICO_H
9
10 #include <string>
11 using namespace std;
12
13 // Estructura que agrupa todo el resultado del diagnóstico.
14 //
15 // NOTA PARA QUIEN ARME main.cpp:
16 // Deben crear una variable de este tipo y llenar los 4 campos
17 // antes de llamar a registrarResultado(). Ejemplo:
18 //
19 // InfoEntorno resultado;
20 // resultado.datoRed = obtenerDatoDeRed(); // de Entorno.h
21 // resultado.datoSistema = obtenerDatoDeSistema(); // de Entorno.h
22 // resultado.aislado = estaEnEntornoAislado(); // de Entorno.h
23 // resultado.detalle = "..."; // describan en texto qué se evaluó
24 //
25 struct InfoEntorno {
26     bool aislado; // true si se detectó entorno aislado, false si no
27     string detalle; // explicación en texto: qué se evaluó y por qué se llegó
28     string datoRed; // resultado de obtenerDatoDeRed() (Entorno.h)
29     string datoSistema; // resultado de obtenerDatoDeSistema() (Entorno.h)
30 };
31
```

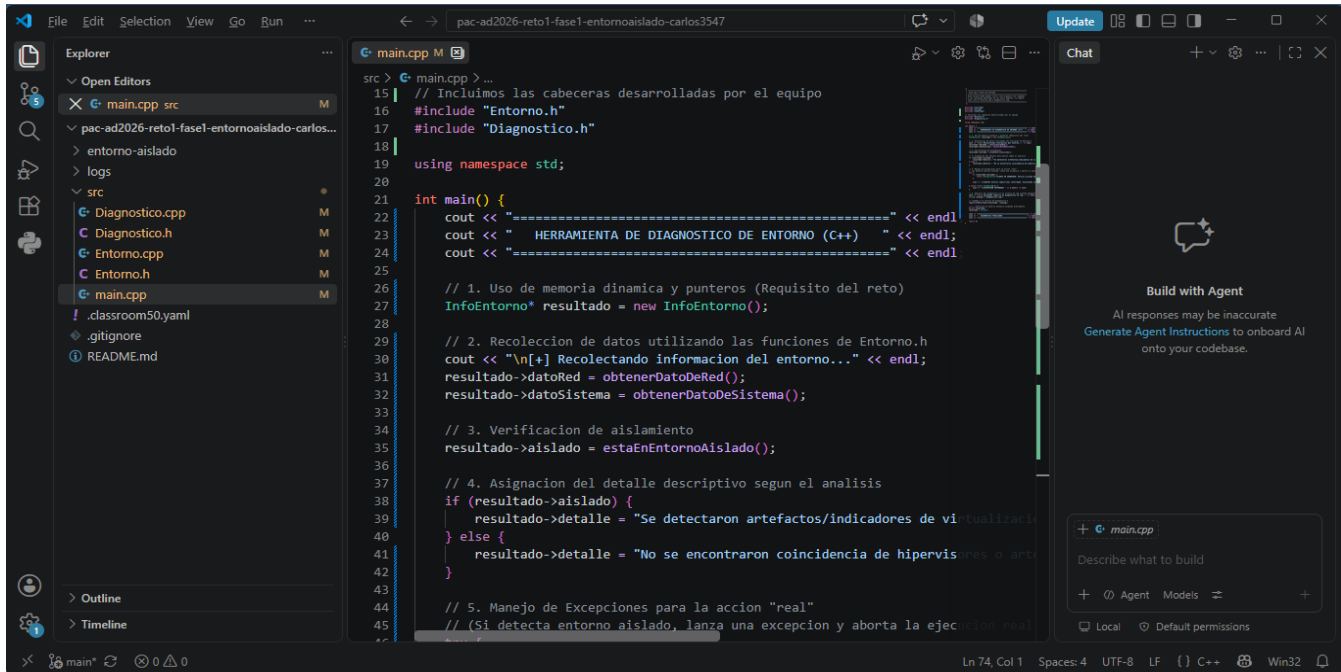


```
1  #include "Diagnostico.h"
2
3  // Escribe el resultado del diagnóstico en el archivo de log indicado.
4  //
5  // NOTA PARA QUIEN ARME main.cpp:
6  // Llámela al final, ya con el struct InfoEntorno completamente
7  // lleno. rutaLog es la ruta del archivo, ej: "logs/diagnostico.log"
8  // (la carpeta logs/ debe existir o la función fallará al abrir
9  // el archivo - eso ya está contemplado con el try/catch interno).
10 //
11 // No necesitan hacer try/catch alrededor de esta llamada: el
12 // manejo de la excepción ya ocurre DENTRO de registrarResultado
13 // (ver Diagnostico.cpp), que atrapa el error y avisa en pantalla.
14 //
15 void registrarResultado(const InfoEntorno& info, const string& rutaLog);
16
17 #endif

```

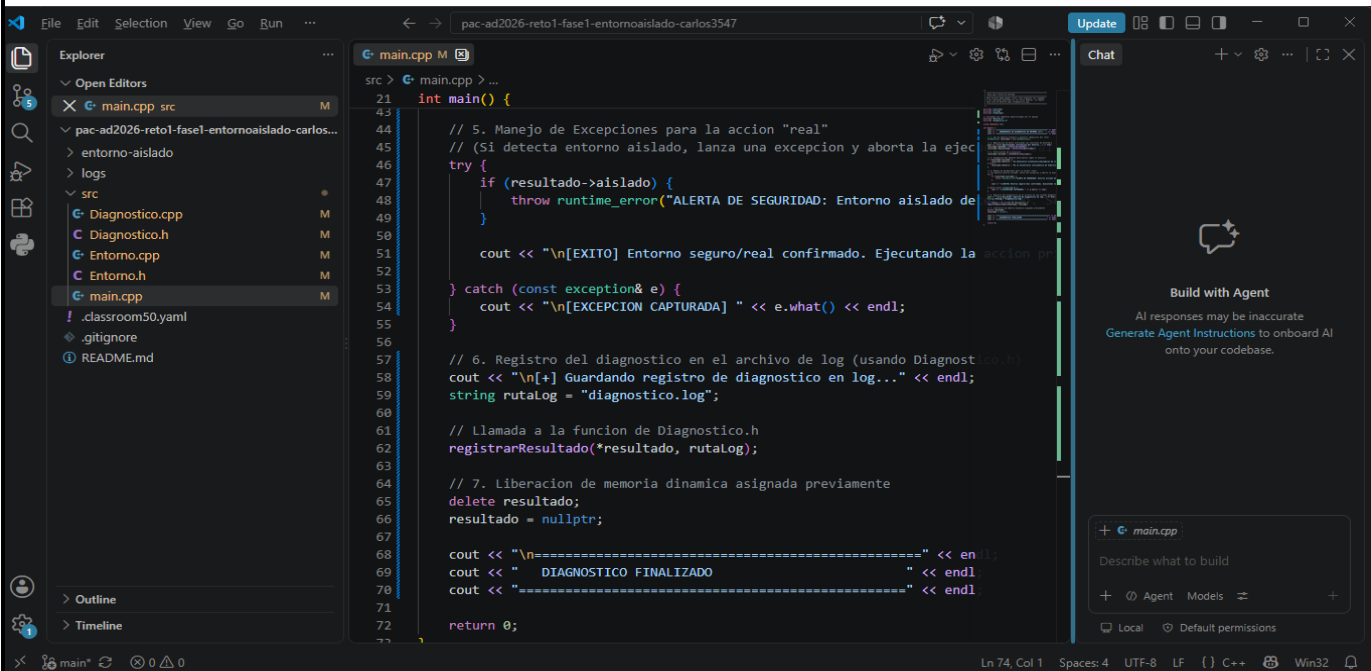
Main.cpp

Se integra el flujo principal del programa combinando los módulos Entorno y Diagnostico.
Se añade la recolección de datos.



```
src > G: main.cpp > ...
15 // Incluimos las cabeceras desarrolladas por el equipo
16 #include "Entorno.h"
17 #include "Diagnostico.h"
18
19 using namespace std;
20
21 int main() {
22     cout << "===== " << endl;
23     cout << "  HERRAMIENTA DE DIAGNOSTICO DE ENTORNO (C++)  " << endl;
24     cout << "===== " << endl;
25
26     // 1. Uso de memoria dinamica y punteros (Requisito del reto)
27     InfoEntorno* resultado = new InfoEntorno();
28
29     // 2. Recoleccion de datos utilizando las funciones de Entorno.h
30     cout << "\n[+] Recoleccion de informacion del entorno..." << endl;
31     resultado->datoRed = obtenerDatoDeRed();
32     resultado->datoSistema = obtenerDatoDeSistema();
33
34     // 3. Verificacion de aislamiento
35     resultado->aislado = estaEnEntornoAislado();
36
37     // 4. Asignacion del detalle descriptivo segun el analisis
38     if (resultado->aislado) {
39         resultado->detalle = "Se detectaron artefactos/indicadores de virtualizacion";
40     } else {
41         resultado->detalle = "No se encontraron coincidencia de hipervisores o arti";
42     }
43
44     // 5. Manejo de Excepciones para la accion "real"
45     // (Si detecta entorno aislado, lanza una excepcion y aborta la ejecucion real)
```

Evaluación del aislamiento, manejo de memoria dinámica con punteros y el control de excepciones para abortar la acción real.



```
21 int main() {
43
44     // 5. Manejo de Excepciones para la accion "real"
45     // (Si detecta entorno aislado, lanza una excepcion y aborta la ejecucion real)
46     try {
47         if (resultado->aislado) {
48             throw runtime_error("ALERTA DE SEGURIDAD: Entorno aislado de");
49         }
50
51         cout << "\n[EXITO] Entorno seguro/real confirmado. Ejecutando la accion pr";
52
53     } catch (const exception& e) {
54         cout << "\n[EXCEPCION CAPTURADA] " << e.what() << endl;
55     }
56
57     // 6. Registro del diagnostico en el archivo de log (usando Diagnostico.h)
58     cout << "\n[+] Guardando registro de diagnostico en log..." << endl;
59     string rutaLog = "diagnostico.log";
60
61     // Llamada a la funcion de Diagnostico.h
62     registrarResultado(*resultado, rutaLog);
63
64     // 7. Liberacion de memoria dinamica asignada previamente
65     delete resultado;
66     resultado = nullptr;
67
68     cout << "\n===== " << endl;
69     cout << "  DIAGNOSTICO FINALIZADO  " << endl;
70     cout << "===== " << endl;
71
72     return 0;
73 }
```


Funcionamiento

La corrección fue agregar `log.fail ()` para detectar si hubo algún problema al escribir el archivo. Lo demás ya funcionaba bien y cumplía con los requisitos, por eso se dejó igual.

```
~/pac-ad2026-reto1-fase1-entornoaislado-carlos3547
Hewlett Packard@DESKTOP-BEM4IC1 CLANG64 ~/pac-ad2026-reto1-fase1-entornoaislado-carlos3547
$ ls -la diagnostico.log
-rw-r--r-- 1 Hewlett Packard Ninguno 224 Aug 30 20:59 diagnostico.log

Hewlett Packard@DESKTOP-BEM4IC1 CLANG64 ~/pac-ad2026-reto1-fase1-entornoaislado-carlos3547
$ ./diagnostico
=====
HERRAMIENTA DE DIAGNOSTICO DE ENTORNO (C++)
=====
[+] Recolectando informacion del entorno...
Dato de sistema:
Dato de red (hostname):
No se encontraron indicadores conocidos de virtualizacion.
[EXITO] Entorno seguro/real confirmado. Ejecutando la accion principal del programa...
[+] Guardando registro de diagnostico en log...
=====
DIAGNOSTICO FINALIZADO
=====

Hewlett Packard@DESKTOP-BEM4IC1 CLANG64 ~/pac-ad2026-reto1-fase1-entornoaislado-carlos3547
$ pwd
/home/Hewlett Packard/pac-ad2026-reto1-fase1-entornoaislado-carlos3547

Hewlett Packard@DESKTOP-BEM4IC1 CLANG64 ~/pac-ad2026-reto1-fase1-entornoaislado-carlos3547
$ |
```

En cuanto al aislamiento lo ejecutamos, pero no nos mostró nada en el log que se ejecuto

